# DFA Simulation in C (with Transition Table)

## Program (dfa.c)

```
#include <stdio.h>
#include <string.h>

int main() {
    int n, m, start, nf, i, j, x, cur, next, idx, k, ok;
    int trans[20][10], final[20] = {0};
    char alpha[10], str[100];

    printf("Number of states (named 0 to n-1): ");
    scanf("%d", &n);
    printf("Number of input symbols: ");
    scanf("%d", &m);
    printf("Input symbols: ");
    for (i = 0; i < m; i++)
        scanf(" %c", &alpha[i]);

    /* read the transition table */
    printf("Enter the transition table:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++) {
            printf("  state %d on '%c' -> ", i, alpha[j]);
            scanf("%d", &trans[i][j]);
        }

    printf("Start state: ");
    scanf("%d", &start);
    printf("Number of final states: ");
    scanf("%d", &nf);
    printf("Final states: ");
    for (i = 0; i < nf; i++) {
        scanf("%d", &x);
        final[x] = 1;
    }

    /* display the transition table */
    printf("\nTransition Table\n");
    printf("State\t");
    for (j = 0; j < m; j++)
        printf("%c\t", alpha[j]);
    printf("\n");
    for (i = 0; i < n; i++) {
        printf("%sq%d%s\t", i == start ? "->" : "  ", i, final[i] ? "*" : " ");
        for (j = 0; j < m; j++)
            printf("q%d\t", trans[i][j]);
        printf("\n");
    }
    printf("(-> start state, * final state)\n");

    /* check strings using the table */
    while (1) {
        printf("\nEnter string to check (or 'quit'): ");
        scanf("%s", str);
        if (strcmp(str, "quit") == 0)
            break;

        cur = start;
        ok = 1;
        printf("q%d", cur);
        for (k = 0; k < (int)strlen(str); k++) {
```

```
            idx = -1;
            for (j = 0; j < m; j++)
                if (alpha[j] == str[k]) { idx = j; break; }
            if (idx == -1) {
                printf("\nInvalid symbol '%c'.\n", str[k]);
                ok = 0;
                break;
            }
            next = trans[cur][idx];
            printf(" --%c--> q%d", str[k], next);
            cur = next;
        }
        if (!ok) {
            printf("String REJECTED\n");
            continue;
        }
        if (final[cur])
            printf("\nString ACCEPTED (ended in final state q%d)\n", cur);
        else
            printf("\nString REJECTED (ended in non-final state q%d)\n", cur);
    }
    return 0;
}
```

## How to run

```
gcc dfa.c -o dfa
./dfa
```

## Sample DFA

```
Example DFA: strings over {0,1} that end with "01"

States      : q0, q1, q2   (start = q0, final = q2)

            0     1
    q0      q1    q0
    q1      q1    q2
    q2      q1    q0
```

## Sample run

```
Number of states (named 0 to n-1): 3
Number of input symbols: 2
Input symbols: 0 1
Enter the transition table:
  state 0 on '0' -> 1
  state 0 on '1' -> 0
  state 1 on '0' -> 1
  state 1 on '1' -> 2
  state 2 on '0' -> 1
  state 2 on '1' -> 0
Start state: 0
Number of final states: 1
Final states: 2

Transition Table
State   0       1
->q0    q1      q0
  q1    q1      q2
```

```
  q2*   q1      q0
(-> start state, * final state)

Enter string to check (or 'quit'): 1101
q0 --1--> q0 --1--> q0 --0--> q1 --1--> q2
String ACCEPTED (ended in final state q2)

Enter string to check (or 'quit'): 110
q0 --1--> q0 --1--> q0 --0--> q1
String REJECTED (ended in non-final state q1)

Enter string to check (or 'quit'): quit
```